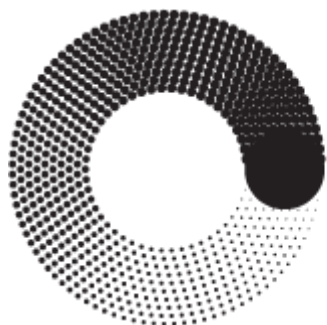


Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования



**МОСКОВСКИЙ
ПОЛИТЕХ**

ОТЧЕТ

по Лабораторной работе 1

по дисциплине: «Технологии компьютерной графики»

Выполнил:	Назаров Т. Д.
Студент Группы:	231-321
Проверили:	Пухова Е.А. Верещагин В.Ю

Москва 2026

1. Цель работы

Разработать веб-приложение для загрузки, просмотра и сохранения изображений с поддержкой стандартных форматов (PNG, JPG) и собственного формата GrayBit-7 (GB7), разработанного для образовательных целей. Приложение должно обеспечивать полный цикл работы с изображениями: декодирование, отображение на canvas, кодирование и сохранение.

2. Постановка задачи

2.1 Основные требования

Приложение должно предоставлять пользователю возможность загружать изображения в трех форматах: PNG, JPG и GB7. Загрузка должна осуществляться двумя способами - через стандартный диалог выбора файла и через перетаскивание файла мышью в специальную область (Drag & Drop).

После загрузки изображение должно быть отображено на элементе canvas. Пользователь должен иметь возможность сохранить обработанное изображение в форматах PNG, JPG или GB7. Для форматов PNG и JPG используется стандартная функциональность браузера, а для формата GB7 требуется написать собственный декодер и кодер.

В нижней части интерфейса должна располагаться строка состояния (status bar), в которой отображается информация о загруженном изображении: ширина в пикселях, высота в пикселях и глубина цвета. Для обычных изображений (PNG, JPG) глубина цвета составляет 8 бит на канал RGB, а для формата GB7 глубина составляет 7 бит на пиксель в оттенках серого.

Приложение должно корректно реагировать на изменение ширины видимой области веб-страницы (viewport). Интерфейс должен оставаться удобным для использования как на больших мониторах, так и на мобильных устройствах.

2.2 Спецификация формата GrayBit-7

Формат GB7 был разработан в образовательных целях для изучения принципов работы с бинарными данными изображений. Это формат без сжатия, использующий 7 бит на пиксель для хранения значения в оттенках серого, при этом восьмой бит может использоваться для хранения информации о маске прозрачности.

Файл формата GB7 состоит из двух основных частей: заголовка и массива пиксельных данных.

Заголовок файла имеет длину 12 байт и начинается с 4-байтовой сигнатуры, которая представляет собой последовательность байтов 0x47 (символ G), 0x42 (символ B), 0x37 (цифра 7) и 0x1D (управляющий символ-разделитель групп). Эта сигнатура однозначно идентифицирует файл как формат GrayBit-7.

После сигнатуры следуют 8 байт мета-информации. Первый байт (смещение 4) содержит версию формата. Текущая версия - 0x01. Второй байт (смещение 5) содержит флаги, где младший бит (бит 0) указывает на наличие маски прозрачности. Остальные биты этого байта зарезервированы для будущего использования и должны быть равны нулю.

Далее следуют два байта (смещение 6), содержащие ширину изображения в пикселях. Ширина хранится в виде 16-разрядного целого числа без знака в порядке от старшего к младшему (сетевой порядок байтов, big-endian). Аналогично, следующие два байта (смещение 8) содержат высоту изображения в том же формате. Последние два байта заголовка (смещение 10) зарезервированы для будущего использования и должны быть равны 0x0000.

Данные изображения начинаются сразу после заголовка (смещение 12). Каждый пиксель кодируется одним байтом. В этом байте младшие 7 бит (биты 0-6) содержат значение оттенка серого в диапазоне от 0 до 127, где 0 соответствует черному цвету, а 127 - белому. Старший бит (бит 7) используется для хранения бита маски, но только в том случае, если в заголовке установлен флаг маски.

Если флаг маски равен нулю, то старший бит каждого пикселя должен игнорироваться и всегда быть равен нулю. Если флаг маски установлен в единицу, то старший бит интерпретируется следующим образом: значение 1 означает, что пиксель является непрозрачным и должен отображаться, значение 0 означает, что пиксель является прозрачным (замаскированным).

Данные изображения хранятся построчно, начиная с верхнего левого угла, затем идут пиксели первой строки слева направо, затем второй строки и так далее. Отступы между строками отсутствуют.

Таким образом, общий размер файла GB7 составляет 12 байт заголовка плюс количество пикселей (ширина умножить на высоту) байт данных.

3. Архитектура приложения

3.1 Структура проекта

noir-graybit-editor/

|— index.html # Главная страница

|— css/

| |— style.css # Стили

|— script.js # Логика приложения

|— README.md # Документация

|— assets/

| |— icons/

| |— logo.png

| |— folder-icon.png

| |— btn-png.png

| |— btn-jpg.png

| |— btn-gb7.png

| |— cursors/ # Курсор, пока настроил только на фаерфокс (на остальных ток svg)

| |— cursor-normal.png

| |— cursor-pointer.png

| |— cursor-crosshair.png

| |— cursor-wait.png

Рисунок 1 - Структура проекта

3.2 Ключевые компоненты

Кодек GrayBit7Codec - класс для преобразования формата GB7:

1. decode(arrayBuffer) - преобразует GB7 в ImageData для отображения на canvas
2. encode(imageData, hasMask) - преобразует ImageData в бинарный формат GB7

Класс DetectiveCase - основное приложение:

1. Загрузка и сохранение файлов
2. Навигация между страницами
3. Отображение информации в статус-баре

3.3 Структура интерфейса

Приложение состоит из трёх экранов:

1. Обложка дела - начальный экран с логотипом и кнопкой открытия
2. Досье (загрузка) - область для выбора файла (кнопка + Drag & Drop)
3. Лаборатория (редактирование) - canvas, кнопки сохранения, статус-бар

4. Реализация формата GB7

4.1 Декодирование (чтение GB7 - ImageData)

```
6      static decode(arrayBuffer) {
7          const bytes = new Uint8Array(arrayBuffer);
8          const view = new DataView(arrayBuffer);
9          for (let i = 0; i < 4; i++) {
10             if (bytes[i] !== this.SIGNATURE[i]) throw new Error('Неверная подпись GB7');
11         }
12         const version = bytes[4];
13         if (version !== this.VERSION) throw new Error(`Неподдерживаемая версия GB7: ${version}`);
14         const flags = bytes[5];
15         const hasMask = (flags & 0x01) !== 0;
16         const width = view.getUint16(6, false);
17         const height = view.getUint16(8, false);
18         const pixelData = new Uint8Array(bytes.buffer, 12, width * height);
19         const imageData = new ImageData(width, height);
20         for (let i = 0; i < width * height; i++) {
21             const pixel = pixelData[i];
22             const gray = pixel & 0x7F;
23             const maskBit = (pixel >> 7) & 0x01;
24             const gray8 = Math.round(gray * (255 / 127));
25             const idx = i * 4;
26             imageData.data[idx] = gray8;
27             imageData.data[idx + 1] = gray8;
28             imageData.data[idx + 2] = gray8;
29             imageData.data[idx + 3] = hasMask ? (maskBit === 1 ? 255 : 0) : 255;
30         }
31         return { imageData, hasMask, width, height };
32     }
33 }
```

Рисунок 2 – Структура кода к 4.1

4.2 Кодирование (ImageData - GB7)

```

33
34     static encode(imageData, hasMask = false) {
35         const width = imageData.width;
36         const height = imageData.height;
37         const data = imageData.data;
38         const buffer = new ArrayBuffer(12 + width * height);
39         const bytes = new Uint8Array(buffer);
40         const view = new DataView(buffer);
41         bytes.set(this.SIGNATURE, 0);
42         bytes[4] = this.VERSION;
43         bytes[5] = hasMask ? 0x01 : 0x00;
44         view.setUint16(6, width, false);
45         view.setUint16(8, height, false);
46         view.setUint16(10, 0, false);
47         for (let i = 0; i < width * height; i++) {
48             const idx = i * 4;
49             const r = data[idx];
50             const g = data[idx + 1];
51             const b = data[idx + 2];
52             const alpha = data[idx + 3];
53             const gray = Math.round(0.299 * r + 0.587 * g + 0.114 * b);
54             const gray7 = Math.round(gray * (127 / 255));
55             let maskBit = 0;
56             if (hasMask) maskBit = alpha >= 128 ? 1 : 0;
57             const pixelByte = (maskBit << 7) | (gray7 & 0x7F);
58             bytes[12 + i] = pixelByte;
59         }
60         return buffer;
61     }
62 }

```

Рисунок 3 – Структура кода к 4.2

5. Пользовательский интерфейс

5.1 Стилистическое решение

Приложение выполнено в нуар-детективном стиле:

1. Цветовая палитра - #0B0D12, #131720, #1B1A1D
2. Акцентный цвет - старая бронза (#D6A56A, #4A3726)
3. Эффекты - пыльная текстура, виньетка, зернистость
4. Курсоры - кастомные детективные курсоры
5. Типографика - Bebas Neue, Oswald, Roboto Condensed

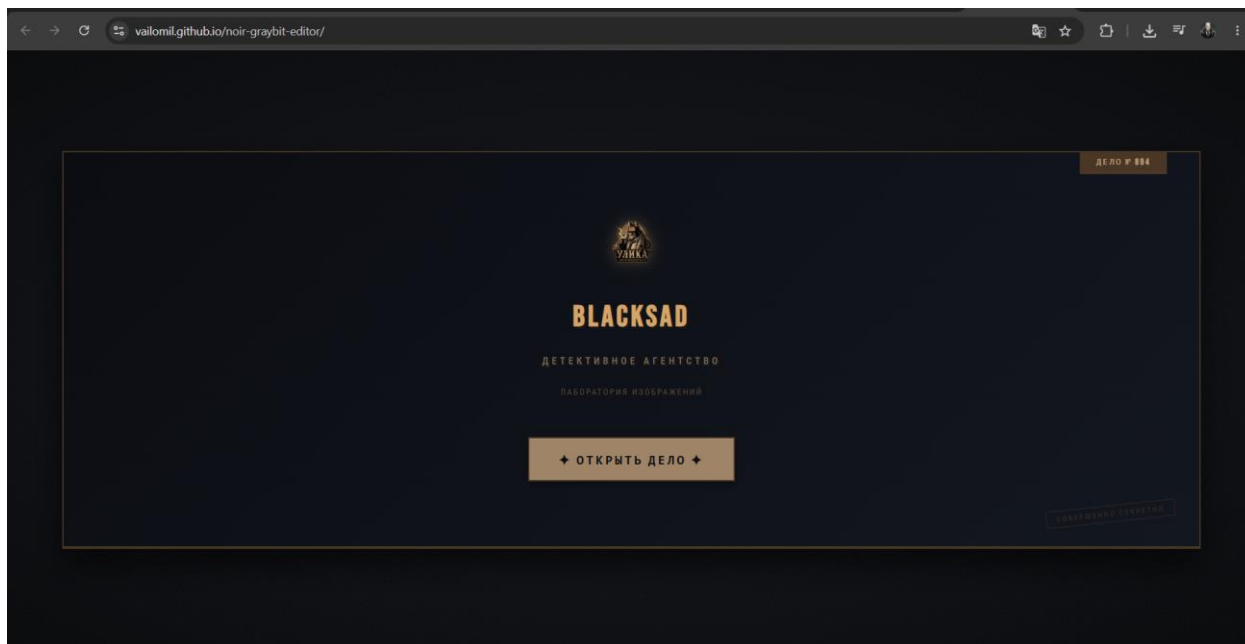


Рисунок 4 – Пример стилистического решения к 5.1

5.2 Страница загрузки (Досье)

1. Область Drag & Drop с визуальной обратной связью
2. Кнопка выбора файла
3. Информация о загруженном файле (имя, формат, размер)

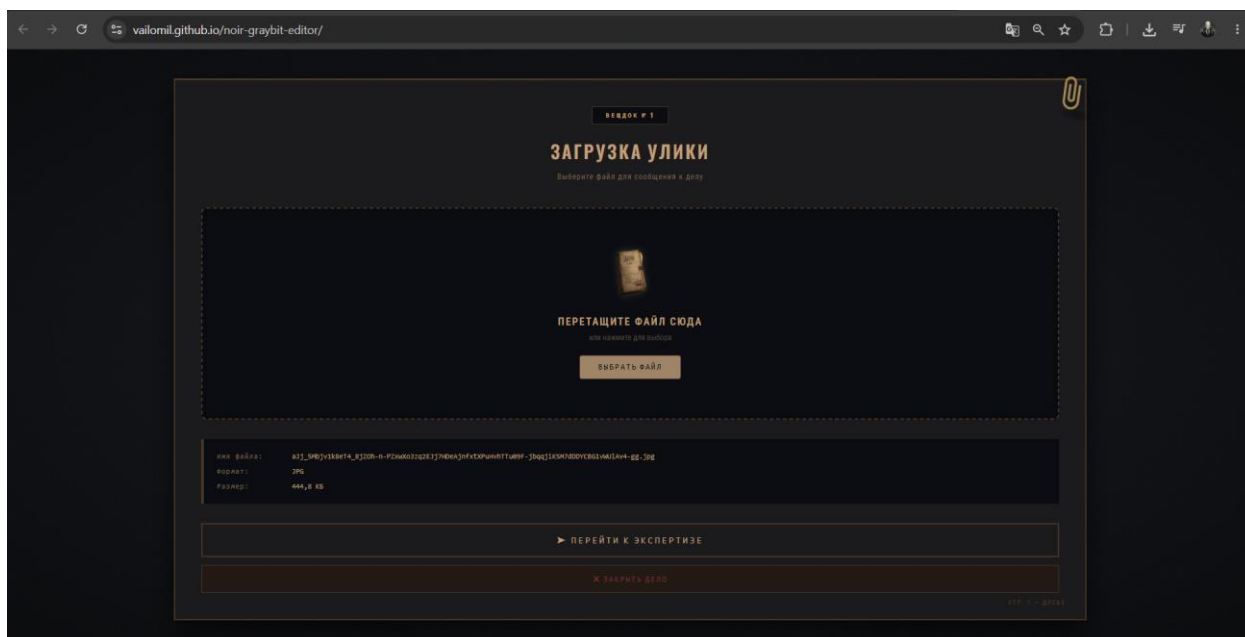


Рисунок 5 – Страница загрузки к 5.2

5.3 Страница редактирования (Лаборатория)

1. Canvas для отображения изображения
2. Кнопки сохранения в PNG, JPG, GB7
3. Статус-бар (размер, глубина, альфа-канал)

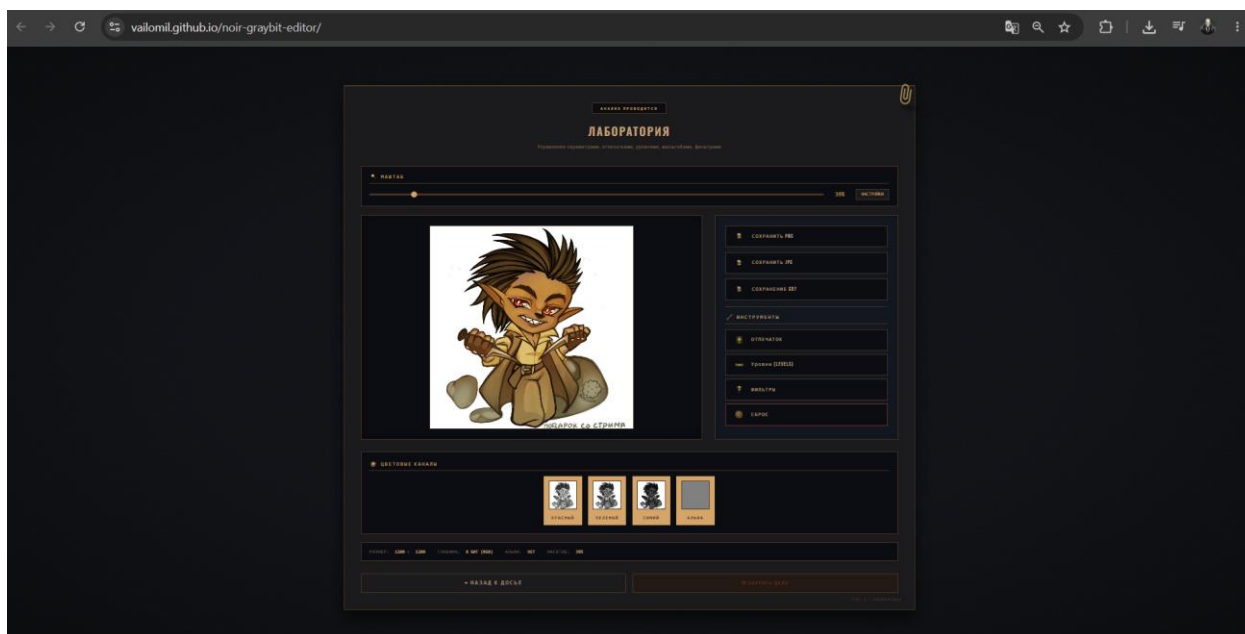


Рисунок 6 – Страница редактирования к 5.3

5.4 Статус-бар

1. Размер - 1920 × 1080 px
2. Глубина - 8 BIT (RGB)
3. Альфа - ЕСТЬ / НЕТ

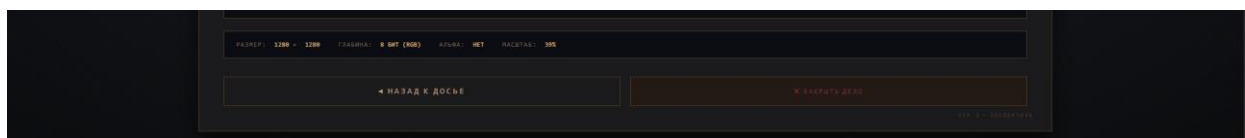


Рисунок 7 – Статус-бар к 5.4

6. Функциональные возможности

6.1 Загрузка файлов

1. Кнопка "ВЫБРАТЬ ФАЙЛ" - стандартный диалог выбора файла
2. Drag & Drop - перетаскивание файла в область загрузки
3. Поддерживаемые форматы - .png, .jpg, .jpeg, .gb7

6.2 Сохранение файлов

1. PNG - полное качество, поддержка прозрачности
2. JPG - сжатие 92%, меньший размер файла
3. GB7 - 7 бит/пиксель, опциональная маска

7. Технические решения

7.1 Используемые технологии

1. HTML - структура страницы
2. CSS - стилизация, адаптив, анимации
3. JavaScript - вся логика (без фреймворков)
4. Canvas API - отображение изображений
5. File API - чтение файлов
6. DataView - работа с бинарными данными GB7

7.2 Обработка бинарных данных GB7

1. Чтение big-endian чисел

```
const width = view.getUint16(6, false);
const height = view.getUint16(8, false);
```

2. Упаковка 7 бит + маска

```
const pixelByte = (maskBit << 7) | (gray7 & 0x7F);
```

3. Извлечение компонентов

```
const gray = pixel & 0x7F;
const maskBit = (pixel >> 7) & 0x01;
```

8. Деплой

8.1 Хостинг

Приложение развернуто на GitHub Pages: <https://VailoMil.github.io/noir-graybit-editor/>

8.2 Настройка

1. Создание репозитория на GitHub
2. Push кода в ветку main
3. Включение GitHub Pages в настройках репозитория
4. Добавление ссылки на демо в файл README.md

9. Заключение

В ходе выполнения лабораторной работы разработано полнофункциональное веб-приложение для работы с изображениями. Особое внимание уделено реализации кодека формата GB7, который полностью соответствует спецификации и корректно обрабатывает 7-битные градации серого и маску прозрачности.

Интерфейс приложения выполнен в уникальном нуар-детективном стиле, что делает работу с инструментом не только функциональной, но и эстетически приятной.

Приложение успешно протестировано на всех целевых форматах, адаптировано под различные размеры экрана и развёрнуто на GitHub Pages для публичного доступа.